

API Documentation

VOW API — Integration Reference & Validation

Turning the client's API overview into our build contract

*The client (David) generated a backend API overview from the VowMade repo. This document does two things: (1) validates it against what the agent actually needs, and (2) restructures it around **how we'll use it** — mapping the real endpoints to our interaction flow, pinning down the Strategy schema, and listing what the grounded registry must hold. This is the artifact the team builds tool-wrappers from and the basis for the backlog.*

1. Verdict — does it make sense?

Yes — and it's a strong result. The document is coherent, well-organised, and was checked against the generated Swagger schema (`/api/openapi`), so it reflects the real contract, not a guess. Most importantly:

The API covers every operation the agent needs, across all milestones. Every stage of our interaction flow maps to a real endpoint (§3). There are no blocking gaps.

Three findings that make our build easier than we assumed:

1. **The Strategy schema is already defined for us.** `POST /api/strategies/` has a concrete request body (§4). The "agree the Strategy schema jointly" task — which we flagged as the single most important early agreement — is now *anchored to the API* rather than invented. Kareem, Wajahat, and Vishal agree on *this* shape.
2. **There's a native draft → approve → activate lifecycle.** Strategies are created in `Draft` status (`POST /strategies/draft/`), then launched via `POST /strategies/{id}/set_status/`. This is a near-perfect fit for our human-approval gate: **the agent builds a draft, the human approves, and only then does** `set_status` activate it. Our "no autonomous spend" gate maps directly onto this.
3. **The pgvector audience feature and forecasting are exposed as endpoints.** `POST /audience-sets/suggest/` ("generate suggested audience groups from a prompt") is almost certainly the award-winning pgvector + OpenAI feature, and there are dedicated `reach-forecast` endpoints. We reuse both by calling them — exactly as planned.

A few things to confirm with David (none blocking — listed in §7): how the *agent* authenticates (the API uses user session/JWT auth), whether `suggest` is the existing pgvector feature,

whether forecasts call Amazon's API or VOW's own model, the idempotency convention for create/activate, and rate limits.

2. How the API works — cross-cutting rules every tool-wrapper must follow

These apply to every call the agent makes, so Wajahat's tool-wrapper layer bakes them in once:

- **Base path:** `/api`.
- **Auth:** Django **session** authentication (cookies) for most endpoints; `/auth` provides token/JWT login (used for partner iframe flows). *How the agent authenticates is the main integration question — see §7.*
- **Advertiser scoping (our multi-tenancy mechanism):** advertiser-scoped endpoints require the `Vowmade-Advertiser-Id` request header (a few use a `selected_advertiser_id` query param instead). **Every scoped tool-wrapper must carry the advertiser context** — this is also how tenant isolation is enforced at the API boundary.
- **Market scoping:** `{market}` path params are country codes (`GB` , `US` , `AU`).
- **Pagination:** page-number style (`count` / `next` / `previous` / `results`), default page size **8**, with `page` / `page_size` params. Wrappers that list must handle paging.
- **Archive pattern:** bulk archive/unarchive take `{ "object_ids": [...] }`.
- **The draft → approve → activate pattern** (see §1, finding 2) — the backbone of our approval flow.
- **Async "suggest" pattern:** `POST /audience-sets/suggest/` kicks off generation and returns an id; `GET /audience-sets/suggest/{id}/` retrieves the result. The wrapper submits, then polls — matches our async design.

3. The agent flow, mapped to real endpoints

This replaces the "representative tool names" on the interaction-flow slide with the **actual** VOW endpoints. This table is the integration contract.

#	Stage	Real VOW endpoint(s)	Returns	Lane (wrapper owner)
1	Understand brief	(agent-side; optional brief-doc upload)	—	Kareem

2	Retrieve audiences	GET /audience s/ · POST /audience - sets/sugg est/ → GET /audience - sets/sugg est/{id}/ · GET /audience - categor ies/ · POST /audience s/{market }/overl ap- ping- audiences / · product- & amc- audiences	audiences with Amazon audience IDs , fees, reach ranges; prompt-suggested groups; overlap/affinity	Vishal (registry) + Kareem
3	Build 3 options (narrow/broad/balanced)	POST /audience -sets/ (create group) · suggest output	audience_ set_id, per-format fees	Kareem
4	Forecast (+ repair loop)	POST /audience - sets/reac	reach, impressions, CPM	Kareem

		h- forecast/ · POST /strategi es/reach- forecast/ · POST /strategi es/{pk}/a udiences/ reach- forecast/		
5	Deals & inventory	GET /deals/? markets=& formats= · GET /deals/fi lter- propertie s/ · GET /rates/ct v/{market }/ · GET /inventor y- sources/	deals with genre, ad- lengths, price/type, media/devic e/location; CTV rate cards; inventory sources	Kareem (deal inference)
6	Validate (guardrails + credit)	GET /credits/ summary/ (budget ≤ account_b alance) · POST /campaign s/canary-	balance; validation result	Kareem + Wajahat (AGT/guardrails)

		check/ (creation prerequisites)		
7	Present & choose	(agent + UI; the strategy card)	—	Kareem / UI
8	Assemble draft strategy	POST /strategies/draft/ (or POST /strategies/ → Draft) · PATCH /strategies/{id}/ · POST /simple-strategies/ for the VOW Stream happy path	draft strategy id, status: "Draft"	Kareem
—	 HUMAN APPROVAL	(the gate — LangGraph interrupt)	—	Wajahat (interrupt)
9	Activate & sync	POST /strategies/{id}/set_status / (launch) → VOW's outbound sync builds campaigns +	live strategy	Wajahat / Kareem

		ad groups on Amazon		
•	Creatives	GET /assets/ POST /assets/a mz_assets /gen_uplo ad_urls/ + /register / GET /creative s/ (pre- approved) · /creative s/recs/ (responsive e- commerce) · /creative s/third- party/	assets & creatives with approval state	Kareem / Basil
•	Tracking (KPIs)	GET /conversi ons/defin itions/ · POST /ad- tags/{mar ket}/even ts/	conversion definitions / events	Kareem
•	Monitoring (M3)	/reports/ campaigns · /ad_group s ·	performance metrics & time-series	(M3)

		/creative s · /audience s · /video · /inventor y · /reports/ metrics_g raph/ · /reports/ performan ce- metrics/		
•	History / audit	GET /strategi es/{pk}/h istory/.. ·	change events (changesets)	Wajahat (audit)

The elegant part: stages 1–8 are all *safe* (drafting, reading, forecasting — no spend). The only spend action is stage 9 (`set_status` → activate), which sits **immediately after the approval gate** — so the API's own lifecycle enforces our safety model.

4. The Strategy schema — our most important shared contract (now concrete)

This is the `POST /api/strategies/` body. It is the Strategy object from the demo, made precise. **This is what Kareem (output), Wajahat (state), and Vishal (grounded fields) agree on — and it's defined by the API, not invented.**

Field	Meaning (mapped to the demo)
<code>name</code>	Strategy name (check uniqueness via <code>/strategies/check_strategy_ name_uniqueness/</code>)

goal	AWARENESS / CONSIDERATION / CONVERSION (the funnel)
primary_currency	e.g. GBP
flight_dates	{ lower, upper } ISO timestamps (the flight)
product_categories , video_product_categories	category IDs (from /strategies/categories/)
product_location	SOLD_ON_AMAZON (endemic) vs off-Amazon
asin_numbers	the ASINs (endemic products)
formats_and_kpis	[{ format, kpi, target_kpi }] — e.g. standard_display + REACH
markets_info	per market: { market, audience_targeting: [{ audience_set_id, audience_type }], budget, currency, base_supply_bid }
assets , pre_approved_creatives , third_party_creatives , rec_creatives	the creatives
market_deals , selected_inventory_sources	deals + inventory
enable_fraud_invalid_traffi c_targeting , enable_brand_safety_targeti ng , content_rating_exclusions , audience_targeting_match_ty pe	brand-safety & targeting controls

conversion_types	tracking/attribution
------------------	----------------------

Response returns the strategy `id` and `status` (starts `Draft`). **The agent's job is to fill this object correctly, ground every ID in it, forecast it, and submit it as a draft.**

5. What the grounded ID registry must hold (for Vishal)

Every identifier the agent puts into a Strategy must be a **real** one, validated against the registry. From the API, the ID types are:

- **Amazon audience IDs** (`amz-audience-id-...`) — from `/audiences/` and inside `audience_groups` . (*The core anti-hallucination target.*)
- **Audience set IDs** (UUIDs) — from `/audience-sets/` (the agent's own grounded groups).
- **ASINs** (`B0...`) — validate via `POST /contextual-targeting/{market}/asin-validation/` .
- **Product-category IDs** (numeric, e.g. `343050382647447279`) — from `/strategies/categories/` and `/contextual-targeting/{market}/product-categories/` .
- **Deal IDs** (`external_deal_id` , e.g. `deal-123`) — from `/deals/` .
- **Conversion event IDs** (`amz_id` , e.g. `event-123`) — from `/conversions/definitions/` .
- **Inventory sources** — from `/inventory-sources/` .

Vishal's registry pulls and keeps these current (scoped per advertiser/market), and Riddhi's hallucination check validates the agent's output against them.

6. Tidied endpoint reference (grouped by how we use it)

The client's doc lists endpoints by Django app; here they are grouped by *our* usage. For exhaustive field-level detail, the original overview and `/api/openapi.yaml` remain the source of truth.

Audiences & forecasting — `/audiences/` , `/audience-categories/` , `/audience-sets/` (+ `suggest/` , `reach-forecast/` , `overlapping-audiences/` , `create_substitute_for_market/`), `/audiences/product-audiences/` , `/audiences/amc-audiences/`

Deals, rates & inventory — `/deals/` , `/deals/filter-properties/` , `/rates/ctv/{market}/` , `/inventory-sources/`

Strategy & execution — `/strategies/` (+ `draft/`, `duplicate/`, `set_status/`, `reach-forecast/`, `categories/`, `check_strategy_name_uniqueness/`, `export/`), `/simple-strategies/`, `/automated-strategies/`, `/campaigns/canary-check/`, and the nested `/strategies/{pk}/campaigns|ad-groups|flight-ranges|creatives|audiences|targeting/...`

Creatives & assets — `/assets/` (+ `gen_upload_urls/`, `register/`, `fix/images/...`), `/creatives/` (pre-approved), `/creatives/recs/` (responsive e-commerce), `/creatives/third-party/`

Targeting — `/contextual-targeting/{market}/products|product-categories|keywords/`, `asin-validation/`, `reach-forecasting/`, `/strategies/locations/{market}/`, postcode validation

Credits — `/credits/`, `/credits/summary/`, `/credits/stripe/`

Tracking — `/conversions/definitions/`, `/ad-tags/{market}/events|ad-tag-events/`

Reporting (M3) —

`/reports/campaigns|ad_groups|creatives|audiences|video|inventory/`, `/reports/metrics_graph/`, `/reports/performance-metrics/`, `/reports/metabase/`

History / audit — `/strategies/{pk}/history/...`

Auth & admin — `/login`, `/auth`, `/google_auth/login/`, `/user`, `/amazon/token/`, `/admin/...`

7. Open questions to confirm with David (short, non-blocking)

1. **Agent authentication (the main one).** The API uses user session auth / JWT (`/auth`, partner iframe). How should the *agent* authenticate — a service account, on-behalf-of the logged-in trader's session, or a JWT? This determines how every tool-wrapper is built.
2. **Is** `/audience-sets/suggest/` the existing pgvector + OpenAI feature? (We're planning to reuse it — confirm it's the same one.)
3. **Do the** `reach-forecast` endpoints call Amazon's forecast API, or VOW's own model? (Answers our last open architecture question and sets Riddhi's accuracy baseline.)
4. **Idempotency for create/activate.** Is there an idempotency-key convention, or does the `draft→activate` flow (create draft once, `set_status` once) give us the safety we need? (Money-safety.)
5. **Rate limits** on the API? (Feeds the tool-call-latency concern.)

6. **Guaranteed (PG) deal creation** — David mentioned it's recently possible via API. Not needed for M1 (we *read* deals), but relevant for later M2.

8. What this unblocks

- The **interaction flow** now has real endpoint names — it's the concrete integration spec.
- **Vishal's registry** sources are confirmed (§5).
- The **Strategy schema** — our most important agreement — is pinned to `POST /strategies/` (§4), so the joint schema session becomes "confirm and extend this," not "design from scratch."
- We can now write the **dependency-ordered backlog** with real, buildable tasks instead of placeholders.

In one line: the API covers everything, the Strategy schema and the human-approval gate map cleanly onto real endpoints (draft → approve → `set_status`), and with the six confirmations above we're ready to turn this into the backlog.